

Table 3: Semantically equivalent substitutes.

Category	Original	Substitutes
Option Names	A.; B.; C.; D.	<i>I.; II.; III.; IV.</i>
		<i>E.; F.; G.; H.</i> <i>1.; 2.; 3.; 4.</i> <i>(a).; (b).; (c).; (d).</i>
Answer Indicator	Answer:	<i>Solution</i>
		<i>Reply</i>
		<i>Response</i>
		<i>Result</i> <i>Choice</i>

BookCorpus BookCorpus (Zhu et al., 2015) is originally introduced to align the books to their movie releases in order to provide rich descriptive explanations for visual content. It contains 74M sentences from various sources of books, and we randomly sample 1000 sentences and also detokenize with Moses.

Random Generated Sentences Here, we follow Xu et al. (2022) and construct 1000 randomly generated sentences. For each sentence, we first sample a random length from 5 tokens to 10 tokens, and then sample the tokens from the whole vocabulary. *All random sentences are generated by uniformly sampled from the vocabulary. All the symbols are equally likely to appear, except the special tokens.*

GSM8K GSM8K (Cobbe et al., 2021) is a dataset of high-quality grade school math problems created by human problem writers. The dataset contains 7500 training problems and 1000 test problems. All the problems are answered with between 2 and 8 steps to solve. It is a frequently used benchmark to evaluate the reasoning ability of large language models (Touvron et al., 2023a; Chowdhery et al., 2022). To analyze the chain-of-thought (Wei et al., 2022) effect of LLMs, we add “Let’s think step by step” followed by the question and right before the answer.

MMLU The MMLU (Hendrycks et al., 2021) dataset is another commonly used benchmark to evaluate the knowledge of large language models. It covers 57 subjects across STEM, the humanities, the social sciences, and more. It ranges in difficulty from an elementary level to an advanced professional level, and it tests both world knowledge and problem-solving ability. We randomly sample 20 test cases from each task, resulting in 1140 test queries in total. In the ‘undesired’ section, we uniformly redistribute the answers and options to isolate the selection bias of LLMs.

B.2 EXPERIMENTAL DETAILS

Random Substitutes Throughout the paper, we adopt random substitutes to isolate the effects of tokens, specific formats, and other components. The random substitutions are conducted in the following manner. To avoid the effect of different sequence lengths, we first tokenize the original sentence or demonstration using the Transformers tokenizer. Then, we replace the part to substitute with random tokens from the corresponding vocabulary. With the substitutes, we exclude the special tokens to ensure that all random tokens are valid.

Self-Reinforcement For each experiment, we report the averaged results across three runs with different seeds. We mainly conduct our experiments over two model families, LLaMA and OPT. For LLaMA models, we use 4 models sized in [7b, 13b, 30b, 65b]. For OPT models, we use 7 models with sizes ranging in [125m, 350m, 1.3b, 2.7b, 6.7b, 13b, and 30b]. Each sentence is repeated 20 times in our experiments. Following Xu et al. (2022), we randomly concatenate a prefix before all repeated sentences.

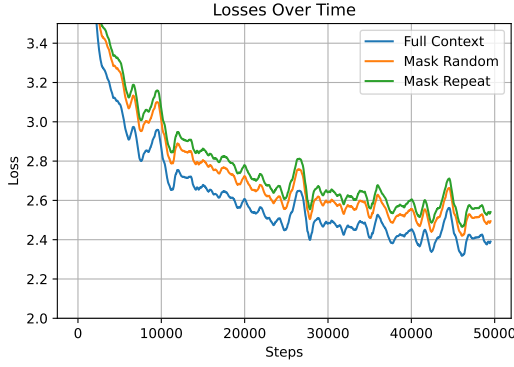


Figure 8: Pretraining losses when masking or not masking the repetitive features.

In-Context Learning In the MMLU experiments, we replace option names and answer indicators to study the importance of token reinforcement in directing output space. Specifically, for each replacement, we choose semantically equivalent substitutes from a pool and randomly replace the original option name/answer indicator with the chosen substitute. In this way, we break the token reinforcement from the demonstrations. We put the pool of our replacement in Table 3.

Significance test We conduct the significance test using paired t-tests, where we randomly split the test set into 5 folds and compute the significance over accuracies on these 5 subsets. In Table 1, we compute the significance levels for [A, B, C] and D separately.

C FURTHER INVESTIGATION OF REASONS AND INNER-WORKINGS FOR TOKEN CO-OCCURRENCE REINFORCEMENT

In Section 3.2, we investigate the reason of token reinforcement as it is inherently embedded within the pretraining corpus. Here, we further investigate token reinforcement with two more experiments, from the learning process and attention mechanism, respectively.

Leveraging repetitive features helps maximizing likelihood. Our next experiment shows that the utilization of repetitive features is a natural choice of LLMs as it helps in the pre-training stage.

To illustrate this, we focus on a conditional language modeling task that predicts tokens based on a given prompt. Formally, we model the probability $P(w_{T_p:T} | w_{<T_p})$, where T and T_p are the total context length and the number of tokens to condition on, respectively.

We then consider three types of prompts. The first one serves as our baseline prompt with $w_{<T_p}$. In the second type, we mask the tokens in the prompt that appear in $w_{T_p:T}$. For the third type, we mask the same quantity of tokens as in the second type, but we select the tokens randomly.

For each type of prompts, we pretrain a language model. The base architecture of our model is the same as LLaMA. Due to limitation of computational cost, we make several changes in hyperparameters for a smaller size. We set the hidden size to 1024 and the FFN size to 4096, and we incorporate 12 layers. The tokenizer is the same as LLaMA and the vocabulary size is 32000. This configuration results in a model with 267M trainable parameters. The dataset we use is the wikipedia-english-2022, and the code base we use is LLaMA-Factory (hiyouga, 2023). We pretrain each model for 50k steps, with a total batch size of 160 sentences per step. Each sentence contains 1024 tokens. Prompt length T_p is $0.75 * 1024 = 768$ tokens. We run our experiments on 4 NVIDIA A100 GPUs, and each experiment takes about 30 hours to finish.

Our results are shown in Figure 8. Masking repetitive features lead to a worse converged loss compared with masking random tokens and no masking. It indicates that repetitive feature is favorable in optimizing training loss, and thus making it natural for language models to use repetitive features.

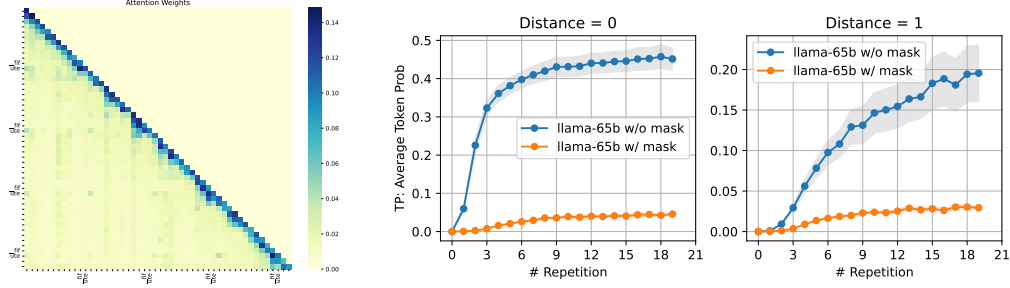


Figure 9: *Left: Attention example of successive token reinforcement.* ‘fif’ and ‘ute’ are the two kept tokens randomly chosen. *Right: Ablation study of attending preceding token.* The results are based on LLaMA-65B.

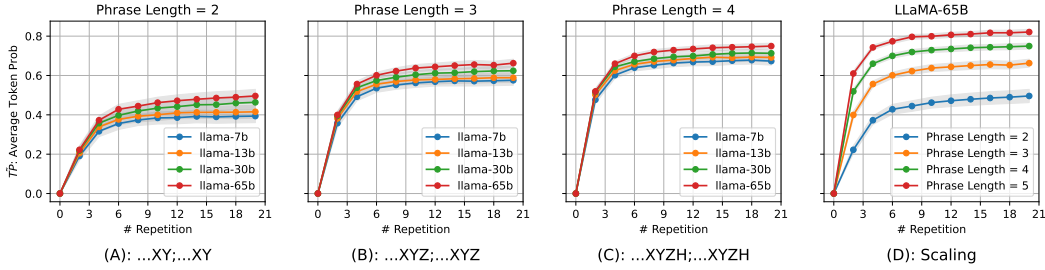


Figure 10: **Phrase co-occurrence reinforcement.** When disrupting the reinforcement loop with random prefixes, the self-reinforcement effect still retains. (A)-(C) plot the scaling of LLaMA models with a certain phrase length (e.g., “XY” denotes length of 2). (D) plots LLaMA-65B’s probability varying phrase length. The gray shadow denotes the standard deviation across 1,000 samples.

Attending preceding token is responsible for reinforcement. Another important question is what inner-working of LLMs is responsible for token reinforcement? We start our investigation with a visualization of attention weights. An example is shown in the left of Figure 9. In this example, each token w_i attends its preceding tokens w_{i-1} with a relatively high attention weights. Thus, our intuitive is that attending preceding token is the key for reinforcing the connection.

To validate this hypothesis, we reproduce the successive reinforcement experiment in Section 3.1, with an attention mask preventing each token by attending its preceding token. Our results are shown in the right of Figure 9. We observe that upon masking the adjacent token, the strength of reinforcement is reduced by a factor of ten. Hence, the reinforcement is not simply realized by attending to similar tokens. Rather, the information is propagated through a process wherein each token iteratively attends to its adjacent token.

D SUPPORTING EXPERIMENTS OF TOKEN CO-OCCURRENCE REINFORCEMENT

In this section, we provide more experimental evidence related to the token co-occurrence reinforcement. The experimental settings in this section follow the same setting as in Section 3.1, except for models to use, datasets to use, and the choices of tokens.

D.1 PHRASE CO-OCCURRENCE REINFORCEMENT

We first construct consecutive tokens to create phrase reinforcement in this section. Specifically, we Without loss of generality, we place the phrase at the end of each demonstration.

We construct a binary mask sequence $m = (\underbrace{0, \dots, 0}_{L_s - L_p}, \underbrace{1, \dots, 1}_{L_p})$, where L_p is the length of the kept phrase. Then, we can define a randomly perturbed sentence $\tilde{s} =$